

Jenkins Q&A

Class no:3

Part 1: Core Concepts & Architecture

Q1. What is Jenkins?

Jenkins is an open-source automation server written in Java. It is used to automate the non-human parts of the software development process, primarily focusing on compiling code, running tests, and deploying software.

Q2. What is CI/CD?

- **CI (Continuous Integration):** The practice of automatically integrating code changes from multiple developers into a shared repository frequently. Each integration is verified by an automated build and test to detect errors quickly.
- **CD (Continuous Delivery/Deployment):** * *Continuous Delivery* ensures code is always in a deployable state, though release to production may require manual approval.
 - *Continuous Deployment* automatically deploys every passing build directly to production without human intervention.

Q3. Why use Jenkins?

- **Open-source and free:** Large community support and zero licensing costs.
- **Extensive Plugin Ecosystem:** Over 1,500+ plugins to integrate with almost any tool in the DevOps toolchain.
- **Distributed Architecture:** Easy to scale by distributing work across multiple machines (Master-Agent architecture).
- **Easy Installation & Configuration:** Platform-independent and configurable via a web interface.

Q4. What is a Jenkins pipeline?

A Jenkins pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins. It allows you to define your entire build process—from source control to deployment—as code.

Q5. What is a Jenkins job?

A Jenkins job (or project) is a user-configured description of work that Jenkins should perform. It could be a simple Freestyle project executing a shell script, or a complex Pipeline project that orchestrates a multi-stage delivery process.

Part 2: Pipelines & Workflow

Q8. What is a Jenkinsfile?

A Jenkinsfile is a text file that contains the definition of a Jenkins Pipeline and is checked into a project's source control repository (like Git). This practice treats the CI/CD workflow as code (**Pipeline as Code**).

Q9. What are stages in Jenkins?

A stage is a block within a pipeline that defines a distinct conceptual part of the CI/CD process. Examples include a "Build" stage, "Test" stage, and "Deploy" stage. It helps visually organize and monitor pipeline execution progress.

Q10. What is a Declarative Pipeline?

A Declarative Pipeline is a modern, structured way to write Jenkins pipelines. It follows a strict, pre-defined schema using a pipeline {} block, making it easier to read, write, and maintain for users without programming experience.

Q11. What is a Scripted Pipeline?

A Scripted Pipeline is the traditional way of writing pipelines in Jenkins using Groovy code. It offers maximum flexibility and power because you can write complex logic, loops, and conditions, but it requires a higher learning curve.

Q21. What is "agent any" in Jenkins?

In a Declarative Pipeline, agent any is an instruction telling Jenkins to allocate any available executor (node or agent) in the environment to execute the pipeline's steps.

Q24. Explain the general Jenkins workflow.

1. **Commit:** A developer pushes code changes to a Git repository.
2. **Trigger:** Git notifies Jenkins via a webhook.
3. **Checkout:** Jenkins provisions an agent, creates a workspace, and pulls the code.
4. **Build & Test:** Jenkins executes the pipeline stages (compiles code, runs unit tests).
5. **Deploy:** If tests pass, Jenkins deploys the artifact to staging or production.
6. **Feedback:** Jenkins sends a success/failure status notification back to the developers (e.g., via Slack or email).

Part 3: Integration & Triggers

Q13. What are plugins in Jenkins?

Plugins are modular add-ons that extend the core functionality of Jenkins. They allow Jenkins to integrate with external tools (like Git, Docker, AWS, SonarQube, and Slack) and customize build behaviors.

Q14. How does Jenkins integrate with Git?

Jenkins integrates with Git via the **Git Plugin**. You provide the Git repository URL and authentication credentials in the job configuration. Jenkins can then pull the source code onto its workspace to run builds.

Q15. What is a webhook?

A webhook is an automated message sent from an app when something happens. In DevOps, a source control management system (like GitHub) sends an HTTP POST request (webhook) to Jenkins to instantly inform it that a new code commit has occurred.

Q16. What is SCM in Jenkins?

SCM stands for **Source Control Management**. In Jenkins, the SCM configuration specifies where the project's source code is hosted (e.g., Git, Subversion, Bitbucket) so Jenkins knows where to fetch the files.

Q18. What is a build trigger?

A build trigger is an event or condition that tells Jenkins to start executing a job. Examples include:

- Pushing code to a repository (Webhooks).
- A time-based schedule (e.g., Nightly builds using Cron syntax).
- The completion of another upstream Jenkins job.

Q26. How does Jenkins trigger automatically after a Git push?

A **Webhook** is configured in the Git repository settings pointing to Jenkins' payload URL. 2. The Jenkins job is configured with the trigger option "**GitHub hook trigger for GITScm polling**" (or equivalent). 3. When a developer pushes code, Git sends a POST request to Jenkins, which instantly activates the job.

Q28. How does Jenkins integrate with Docker?

- Jenkins integrates with Docker using plugins (like the **Docker Pipeline Plugin**). This allows Jenkins to:
- Spin up dynamic Docker containers to act as clean build agents.
- Build custom Docker images from Dockerfiles during the pipeline.
- Push newly built images directly to registries like Docker Hub or AWS ECR.

Part 4: Administration & Troubleshooting

Q19. What is a Jenkins workspace?

A workspace is a specific, dedicated directory on the local file system of a Jenkins node/agent where individual jobs perform their work. This is where source code is cloned and build artifacts are generated.

Q20. What is an environment variable in Jenkins?

Environment variables are global key-value pairs stored by Jenkins that inject contextual information into build scripts. Examples include built-in variables like `${BUILD_NUMBER}` and `${JOB_NAME}`, or custom variables defined by the administrator.

Q22. How do you restart Jenkins?

- Jenkins can be restarted via its URL path by logging in as an administrator and typing:
- `http://[jenkins-url]/restart` (Forces an immediate restart).
- `http://[jenkins-url]/safeRestart` (Waits for all running jobs to complete before restarting).
- Alternatively, via command line: `sudo systemctl restart jenkins`.

Q23. Where are Jenkins logs stored?

- **UI Access:** Inside Jenkins under **Manage Jenkins > System Log**.
- **Linux/Ubuntu OS:** Typically located at `/var/log/jenkins/jenkins.log`.
- **Windows OS:** Located in the installation directory under `jenkins.err.log` and `jenkins.out.log`.

Q25. If a Jenkins build fails, what steps will you take to check it?

1. **Check Console Output:** Review the detailed step-by-step logs of the failing build to pinpoint the exact line or error code that failed.
2. **Review Recent Code Changes:** Check the SCM changes attached to the build to see what code was modified.
3. **Verify Node/Agent Status:** Ensure the build agent hasn't run out of disk space, memory, or lost connection to the master.
4. **Validate Tool Paths:** Check if the required compiler versions, SDKs, or environmental dependencies are properly configured on the machine.